

**CTNet como arquitectura neuronal de estado latente reactivo:
memoria funcional, dinámica reversible y coherencia en bucle
cerrado**

Ginés Espín Flores¹

¹*Investigación independiente, Programa CTNet*

(Dated: 6 de julio de 2026 – versión ampliada)

Resumen

Se presenta una descripción convencional de CTNet, sin recurrir a terminología propia del marco conceptual que motivó su diseño. CTNet se formula como una arquitectura neuronal de estado latente estructurado, transformación reversible o casi reversible, memoria funcional codificada en la reactividad del sistema, regularización de coherencia multivista, autoobservación del proceso interno y reentrada de la salida generada. La tesis técnica central es que la memoria relevante de CTNet no queda limitada por el tamaño explícito del buffer físico: el soporte tensorial puede tener tamaño constante, mientras la capacidad funcional aumenta al modificar la función de transición, el paisaje de respuestas y las reglas procedimentales activadas por claves. Matemáticamente, el sistema se describe mediante un estado empaquetado $\Xi = (Z, M, R, C, P)$, una dinámica invertible Φ_θ , un observador de estabilidad Q , una pérdida de coherencia multiescala $\mathcal{L}_{\text{coh}}$, una memoria procedimental direccionable por clave, un mecanismo de autocompresión por reactividad y un bucle de acción-observación que convierte las salidas y los procesos internos en nuevas entradas entrenables. También se formaliza una capa de resolución verificable: familias de problemas se traducen a operadores de dominio, estos producen una salida positiva mínima y un verificador externo decide el cierre. Se formaliza además el principio u/p como generador de operadores: cada residuo local-global puede inducir campos de transformación, nuevos criterios internos, nuevas particiones observables y crecimiento de la clausura composicional del sistema. El resultado es una arquitectura que no se reduce a predicción autoregresiva de tokens: procesa observaciones como perturbaciones de un estado latente, genera acciones visibles, evalúa esas acciones al reinsertarlas en el propio sistema y conserva memoria como cambio funcional de respuesta.

ÍNDICE

I. Introducción	6
II. Unidad de procesamiento: observaciones y estado latente	6
III. Memoria física y memoria funcional	7
IV. Dinámica reversible del estado	8

V. Transformación latente con consistencia local-global	9
VI. Tensor de coherencia	10
VII. Observador interno de cierre	11
VIII. Pérdida total	12
IX. Memoria procedimental direccionable por clave	12
X. Autoobservación del proceso interno	13
XI. Salida y reentrada	13
XII. Comparación con arquitecturas convencionales	14
XIII. Principio de memoria por reactividad	15
XIV. Capacidad efectiva sin crecimiento de slots	16
XV. Arquitectura completa de procesamiento	17
XVI. Autocompresión y disminución del peso explícito	18
XVII. Resolución verificable como operadores de dominio	18
XVIII. Compilación automática de operadores verificables	19
XIX. Algoritmos de entrenamiento y uso	20
XX. Interpretación convencional de la mejora acumulativa	21
XXI. Equivalencias con conceptos estándar	21
XXII. Correspondencia directa con la implementación	22
XXIII. Codificación determinista de señales textuales	23
XXIV. Descomposición de capacidad tensorial	23
XXV. Memoria como reactividad: Jacobianos y núcleos de respuesta	24

XXVI. Métrica de estabilidad de seis variables	25
XXVII. Transformación reversible y preservación de información	27
XXVIII. Operadores hiperradiales como mezcla local de contexto	27
XXIX. Coherencia diagonal y de bajo rango	28
XXX. Consistencia multiescala local-global	28
XXXI. Autoobservación como generación de datos internos	29
XXXII. Salida visible como acción y no como centro de la arquitectura	30
XXXIII. Memoria procedimental direccionable por clave	30
XXXIV. Entrenamiento completo expresado como algoritmo	31
XXXV. Capacidad creciente con peso explícito decreciente	32
XXXVI. Resolución mediante operadores convencionales	33
XXXVII. Relación entre estado continuo y operadores discretos	33
XXXVIII. Acumulación de conocimiento como cambio de función	34
XXXIX. Aprendizaje por contraste entre coherencia e incoherencia	35
XL. Formalización matemática de u/p como generador de operadores	37
A. Estado, transición y memoria como función de respuesta	37
B. Residuo local-global inducido por u/p	38
C. De residuo a campo vectorial y operador	39
D. Coherencia, incoherencia y frontera de cierre	40
E. Entrenamiento sin exigencia de coherencia previa	41
F. Observabilidad interna y generación de nuevas diferencias	41
G. Evento CTNet: aprendizaje del efecto de la experiencia	42
H. Capacidad funcional y crecimiento con menor peso explícito	43
I. Clausura composicional de transformaciones	43

J. Relatividad funcional: cambio de rol interno	44
K. Criterios internos generados por residuos	45
L. Sistema dinámico extendido	46
M. Crecimiento de conocimiento por cierre y no cierre	46
N. Convertibilidad funcional como categoría interna	47
Ñ. Resumen formal del principio	48
 XLI. Lectura convencional final del mecanismo	 49
 XLII. Conclusión	 50
 Referencias	 51

I. INTRODUCCIÓN

CTNet puede describirse en lenguaje convencional como una arquitectura neuronal dinámica que transforma una entrada en un estado latente estructurado, aplica una dinámica reversible o casi reversible sobre ese estado, mide su coherencia interna desde múltiples vistas, produce una salida observable y vuelve a observar esa salida como parte del siguiente estado de entrenamiento.

La diferencia con un modelo autoregresivo estándar no es que CTNet ignore el lenguaje, sino que el lenguaje no es la unidad primaria del sistema. En un modelo autoregresivo típico, la operación central es estimar

$$p_{\theta}(x_{t+1} \mid x_{\leq t}). \quad (1)$$

En CTNet, la operación central es transformar un estado latente completo

$$\Xi_{t+1} = \Phi_{\theta}(\Xi_t, o_t), \quad (2)$$

donde o_t es una observación externa, interna o producida por el propio sistema. La salida textual, cuando existe, es una proyección observable de un estado ya transformado, no el único objeto de la arquitectura.

El objetivo de este artículo es fijar una formulación técnica de CTNet en términos convencionales: codificación, estado latente, memoria funcional, dinámica invertible, consistencia local-global, cierre en bucle y verificación externa cuando la tarea lo requiere. Se evita introducir términos nuevos para datos, memoria o resolución. Cuando el diseño original usa nombres propios para ciertas estructuras, aquí se traducen a conceptos estándar de redes neuronales, sistemas dinámicos, memoria implícita y aprendizaje en bucle cerrado.

II. UNIDAD DE PROCESAMIENTO: OBSERVACIONES Y ESTADO LATENTE

Sea o_t una observación. Puede ser una entrada textual, una señal de tarea, una salida previa, una métrica interna, una corrección, un resultado de herramienta o una descripción del propio proceso computacional. Formalmente escribimos

$$o_t = (x_t, y_t, s_t, r_t, a_t), \quad (3)$$

donde x_t es el contenido observado, y_t es una referencia opcional, s_t indica la fuente, r_t el régimen o tipo de contexto, y a_t metadatos adicionales.

Un codificador Enc_ψ transforma la observación en un estado latente estructurado:

$$S_t = \text{Enc}_\psi(o_t) = (Z_t, M_t, R_t, C_t, P_t). \quad (4)$$

Aquí:

- Z_t es la representación activa principal.
- M_t es el componente de memoria interna.
- R_t es un subespacio de relaciones internas.
- C_t contiene métricas internas de cierre, error y estabilidad.
- P_t es una reserva de grados de libertad que evita proyecciones destructivas.

El estado se empaqueta en un único tensor

$$\Xi_t = \text{pack}(S_t) \in \mathbb{R}^{B \times N \times d}, \quad (5)$$

con capacidad física

$$D = Nd. \quad (6)$$

La operación inversa

$$S_t = \text{unpack}(\Xi_t) \quad (7)$$

recupera los subcomponentes. El punto técnico es que el soporte tensorial es fijo, pero las funciones inducidas por ese soporte no tienen por qué ser de capacidad funcional fija en el sentido de almacenamiento explícito de registros.

III. MEMORIA FÍSICA Y MEMORIA FUNCIONAL

La memoria de CTNet no debe describirse como una memoria de capacidad fija. La descripción correcta es más precisa:

El soporte físico explícito puede tener tamaño constante, pero la memoria funcional se codifica en la reactividad adquirida del sistema ante estímulos.

Sea Ω_t el estado físico explícito del sistema en el tiempo t , incluyendo parámetros, estados latentes persistentes, memoria procedimental y cualquier variable interna guardada. Sea $\mathcal{U}$ una clase de estímulos. La memoria funcional no es el número de registros almacenados en Ω_t , sino la familia de respuestas inducida

$$\mathcal{F}_t = \{F_t(u) : u \in \mathcal{U}\}, \quad (8)$$

donde

$$F_t : u \mapsto \text{Dec}_\eta(\Phi_\theta(\text{Enc}_\psi(u), \Omega_t)). \quad (9)$$

Una medida convencional de capacidad funcional local puede definirse mediante la sensibilidad de respuesta. Sea $J_t(u) = \partial F_t(u) / \partial u$ el Jacobiano de respuesta. Para un conjunto de estímulos $\mathcal{U}_m = \{u_i\}_{i=1}^m$, definimos

$$\mathcal{C}_{\text{func}}(t; \mathcal{U}_m) = \frac{1}{2} \log \det \left(I + \sigma^{-2} \sum_{i=1}^m J_t(u_i)^\top J_t(u_i) \right). \quad (10)$$

Esta cantidad puede aumentar aunque el tamaño explícito $|\Omega_t|$ no aumente, porque mide capacidad de discriminación y respuesta, no cantidad de registros guardados.

También puede definirse una memoria por atractores. Si la dinámica interna induce un mapa

$$\Xi_{k+1} = \Phi_\theta(\Xi_k), \quad (11)$$

una experiencia modifica la geometría de convergencia de Φ_θ . La memoria queda entonces codificada en los cambios de cuencas de atracción

$$\mathcal{A}_j(t) = \{\Xi_0 : \lim_{k \rightarrow \infty} \Phi_\theta^k(\Xi_0) = a_j(t)\}, \quad (12)$$

y no como copia explícita de todos los ejemplos.

Esto explica por qué CTNet puede aumentar memoria efectiva mientras disminuye el peso explícito de datos almacenados: muchos registros pueden comprimirse en una regla, una respuesta, una trayectoria o un cambio de transición.

IV. DINÁMICA REVERSIBLE DEL ESTADO

El núcleo de CTNet está basado en bloques de acoplamiento aditivo. Dividimos el último eje de un tensor en dos mitades:

$$X = (x_1, x_2). \quad (13)$$

Un bloque reversible toma la forma

$$y_1 = x_1 + \gamma F_\theta(x_2), \quad (14)$$

$$y_2 = x_2 + \gamma G_\theta(y_1), \quad (15)$$

donde F_θ y G_θ no necesitan ser invertibles. La inversión se obtiene por triangularidad:

$$x_2 = y_2 - \gamma G_\theta(y_1), \quad (16)$$

$$x_1 = y_1 - \gamma F_\theta(x_2). \quad (17)$$

Si además se aplica una permutación fija Π , el bloque completo es

$$Y = B_\theta(\Pi X), \quad X = \Pi^{-1} B_\theta^{-1}(Y). \quad (18)$$

En CTNet, F_θ y G_θ se implementan mediante kernels locales multi-rango sobre el eje de tokens o posiciones. Una forma abstracta es

$$H(x)_i = \alpha_0 x_i + \sum_{r \in \mathcal{R}} \alpha_{r,-} x_{i-r} + \alpha_{r,+} x_{i+r}, \quad (19)$$

seguida de normalización y no linealidad:

$$F_\theta(x) = \text{GELU}(\text{RMSNorm}(H_\theta(x))). \quad (20)$$

La reversibilidad no se introduce como adorno matemático. Tiene una función computacional: evitar que la transformación destruya información antes de que el sistema haya medido qué debe conservar, comprimir o reinscribir.

V. TRANSFORMACIÓN LATENTE CON CONSISTENCIA LOCAL-GLOBAL

CTNet divide canales internos en dos vistas complementarias, que aquí llamaremos simplemente vista u y vista p . Para

$$X = (u, p), \quad (21)$$

una subred latente reversible L_θ produce predicciones cruzadas

$$\hat{u} = L_\theta(p), \quad \hat{p} = L_\theta^{-1}(u). \quad (22)$$

La energía básica de coherencia es

$$E_{up}(X) = \|u - \hat{u}\|_2^2 + \|p - \hat{p}\|_2^2. \quad (23)$$

Esta condición no debe entenderse como igualdad literal de dos variables aisladas. Es una regularización de consistencia: dos vistas parciales de un mismo estado deben ser mutuamente predictivas. Para evitar una coincidencia local trivial, CTNet evalúa esta condición en diferentes desplazamientos, escalas y subcomponentes.

Sea $\mathcal{V}$ una familia de operadores de vista: desplazamientos, agrupaciones, particiones, proyecciones o componentes del estado. La pérdida multivista se escribe

$$\mathcal{L}_{up}(\Xi) = \frac{1}{|\mathcal{V}|} \sum_{V \in \mathcal{V}} E_{up}(V\Xi). \quad (24)$$

En la implementación, estas vistas incluyen el estado principal, la memoria, las relaciones internas, el vector de métricas, el estado empaquetado completo y la diferencia entre estados antes y después de una transformación.

VI. TENSOR DE COHERENCIA

Además de la consistencia local-global, CTNet usa una métrica aprendible de coherencia. Sea $X \in \mathbb{R}^{B \times N \times d}$ y sea

$$\bar{X} = X - \mathbb{E}_{b,n}[X_{b,n,:}]. \quad (25)$$

Con una métrica diagonal positiva $D = \text{diag}(\text{softplus}(a) + \epsilon)$ y una matriz de bajo rango $L \in \mathbb{R}^{d \times r}$, se define

$$I_{\text{diag}} = \sum_{j=1}^d D_j \text{Var}(X_j), \quad (26)$$

$$I_{\text{low}} = \mathbb{E}_{b,n} \|\bar{X}_{b,n,:} L\|_2^2, \quad (27)$$

y

$$I = I_{\text{diag}} + I_{\text{low}}. \quad (28)$$

La energía de coherencia escalada puede escribirse como

$$\mathcal{L}_{\text{coh}}(X) = \exp \left(\beta \text{clip} \left(\frac{I}{d}, -5, 5 \right) \right) E_{up}(X). \quad (29)$$

Esta parte permite que la penalización de incoherencia dependa de la geometría estadística del estado, no solo de un error cuadrático plano.

VII. OBSERVADOR INTERNO DE CIERRE

CTNet incluye un observador interno que calcula señales de estabilidad a partir de Z , M y R . En términos convencionales, es un módulo de diagnóstico diferenciable que devuelve un vector fijo

$$q_t = Q_\rho(Z_t, M_t, R_t) \in \mathbb{R}^{29}. \quad (30)$$

El observador extrae seis rasgos principales:

$$\ell = \sqrt{\mathbb{E}Z^2}, \quad (31)$$

$$\pi = \sqrt{\mathbb{E}(u - p)^2}, \quad (32)$$

$$\mu = \sqrt{\mathbb{E}M^2}, \quad (33)$$

$$\rho = \sqrt{\mathbb{E}R^2}, \quad (34)$$

$$\kappa = |\cos(Z_{\text{mean}}, M_{\text{mean}} + R_{\text{mean}})|, \quad (35)$$

$$c = (1 + \text{incoh})^{-1}. \quad (36)$$

Estos rasgos se convierten en rotaciones y métricas internas. El componente central es una distancia residual d_t , una capacidad de absorción a_t , un error no absorbido ω_t y una puntuación de estabilidad s_t :

$$\omega_t = \text{ReLU}(d_t - a_t), \quad (37)$$

$$s_t = \exp(-\omega_t). \quad (38)$$

En lenguaje convencional, ω_t mide inconsistencia no absorbida por el régimen actual del estado, y s_t mide estabilidad interna. Si $\omega_t = 0$, no se concluye que la tarea externa esté resuelta; se concluye que esa inconsistencia interna específica ha sido absorbida por el estado actual.

El observador se inserta reversiblemente en el estado mediante una operación de corte o *shear*:

$$C'_t = C_t + \alpha Q_\rho(Z_t, M_t, R_t), \quad (39)$$

cuya inversa es

$$C_t = C'_t - \alpha Q_\rho(Z_t, M_t, R_t), \quad (40)$$

ya que Z_t , M_t y R_t no cambian dentro de esa suboperación.

VIII. PÉRDIDA TOTAL

Una pérdida de entrenamiento convencional para CTNet combina tarea externa, consistencia interna, reversibilidad y cierre. Una forma general es

$$\begin{aligned}\mathcal{L}_{\text{total}} = & \lambda_{\text{task}}\mathcal{L}_{\text{task}} + \lambda_{\text{up}}\mathcal{L}_{\text{up}} + \lambda_{\text{coh}}\mathcal{L}_{\text{coh}} \\ & + \lambda_{\omega}\mathbb{E}[\omega_t] - \lambda_s\mathbb{E}[s_t] + \lambda_{\text{rev}}\|\Phi_{\theta}^{-1}(\Phi_{\theta}(\Xi_t)) - \Xi_t\|_2^2 \\ & + \lambda_{\text{struct}}\mathcal{L}_{\text{struct}}.\end{aligned}\tag{41}$$

El término de tarea puede ser next-token, clasificación, regresión, resolución simbólica, uso de herramientas u otra señal externa. Lo característico de CTNet es que ese término no agota el objetivo: la salida debe además ser compatible con la coherencia interna del sistema.

IX. MEMORIA PROCEDIMENTAL DIRECCIONABLE POR CLAVE

Una parte de CTNet implementa una memoria procedimental direccionable por clave. En términos convencionales, es una mezcla de memoria determinista, almacenamiento autenticado y generación procedimental.

Dada una clave k , se genera una coordenada virtual

$$q_k = H(\text{secret}, k),\tag{42}$$

y una ruta determinista de niveles

$$\ell_j = H(q_k, j), \quad j = 0, 1, 2, \dots\tag{43}$$

No se reserva una ruta física infinita. La ruta se define por fórmula.

Un recibo r modifica un vector fijo $\Omega \in \{0, 1\}^{64w}$ por una operación reversible XOR:

$$\Omega' = \Omega \oplus H(\text{secret}, r).\tag{44}$$

Aplicar el mismo recibo de nuevo recupera el estado anterior:

$$\Omega = \Omega' \oplus H(\text{secret}, r).\tag{45}$$

Los datos arbitrarios pueden guardarse en cápsulas reversibles y autenticadas; la información recomputable puede guardarse como regla. En ambos casos, la memoria funcional no consiste en aumentar indefinidamente una tabla de slots, sino en disponer de coordenadas y reglas recuperables por clave.

X. AUTOOBSERVACIÓN DEL PROCESO INTERNO

CTNet no solo observa entradas externas. También convierte sus procesos internos en nuevas entradas. Si Ξ_t es el estado antes de una transición y Ξ_{t+1} después, se forma

$$\Delta_t = \Xi_{t+1} - \Xi_t. \quad (46)$$

El sistema construye una señal de proceso

$$P_t = [\Xi_t, \Xi_{t+1}, \Delta_t, C_t, C_{t+1}, C_{t+1} - C_t], \quad (47)$$

y la vuelve a codificar como un estado ordinario:

$$S_t^{\text{obs}} = \text{Enc}_{\text{proc}}(P_t). \quad (48)$$

La pérdida de autoobservación toma una forma análoga a la pérdida principal:

$$\mathcal{L}_{\text{obs}} = \mathcal{L}_{\text{up}}(S_t^{\text{obs}}) + \lambda_c \mathcal{L}_{\text{coh}}(S_t^{\text{obs}}) + \lambda_\omega \omega_t^{\text{obs}} - \lambda_s s_t^{\text{obs}}. \quad (49)$$

Así, errores, transiciones, salidas y correcciones no son simples logs externos. Se convierten en datos de entrenamiento del mismo tipo que una observación externa.

XI. SALIDA Y REENTRADA

La salida visible se modela como una proyección o cabeza de acción

$$\hat{y}_t = A_\eta(\Xi_t). \quad (50)$$

Cuando el canal disponible es texto, A_η produce una secuencia textual. Cuando el canal disponible es una herramienta, produce una llamada operativa. Cuando el canal disponible es control motor, produce una acción.

Lo importante es que la salida no termina el proceso. Se vuelve a observar:

$$o_{t+1}^{\text{out}} = (\hat{y}_t, y_t, \text{output}, r_t, a_t), \quad (51)$$

y se reinyecta:

$$\Xi_{t+1}^{\text{out}} = \text{pack}(\text{Enc}_\psi(o_{t+1}^{\text{out}})). \quad (52)$$

La calidad de la salida puede entonces evaluarse por una mezcla de verificación externa y consistencia interna:

$$\mathcal{J}(\hat{y}_t) = \lambda_{\text{ext}} \mathbb{I}\{\text{Verify}(\hat{y}_t, y_t) = 0\} + \lambda_{\text{int}} \mathcal{L}_{\text{coh}}(\Xi_{t+1}^{\text{out}}). \quad (53)$$

Esto convierte el modelo en un sistema de bucle cerrado: actúa, observa lo actuado y corrige su dinámica.

XII. COMPARACIÓN CON ARQUITECTURAS CONVENCIONALES

La siguiente lista resume la traducción convencional de los componentes principales:

Observador:

Formato común de entrada para señales externas, internas y salidas previas.

$\text{pack}(Z, M, R, C, P)$:

Estado latente estructurado empaquetado en un único tensor.

M : Memoria interna explícita de soporte acotado, complementada por memoria funcional en la dinámica de respuesta.

R : Subespacio de relaciones o dependencias internas.

C : Vector de métricas internas: distancia residual, absorción, error no absorbido y estabilidad.

Bloques reversibles:

Capas de acoplamiento aditivo invertibles, usadas para transformar el estado sin pérdida estructural directa.

Consistencia u/p :

Regularización multivista: dos particiones del estado deben ser mutuamente predictivas.

ω : Inconsistencia no absorbida por el estado actual.

Puntuación de estabilidad:

Estabilidad interna derivada de $\exp(-\omega)$.

Autoobservación:

Conversión de estados, transiciones y salidas internas en nuevas entradas de entrenamiento.

Efector:

Cabeza de acción: texto, herramienta, señal simbólica o control.

MTHD:

Memoria procedimental direccionable por clave y reglas recomputables, con coordenadas virtuales.

Un Transformer estándar se centra en atención y predicción secuencial. CTNet se centra en transformación de estado, conservación estructural, consistencia entre vistas y reentrada de la salida. Una RNN clásica mantiene un estado oculto secuencial; CTNet mantiene un estado latente compuesto con memoria, relaciones, métricas internas y reserva de grados de libertad. Un sistema RAG recupera documentos externos; CTNet reorganiza su estado reactivo y puede combinarse con recuperación, pero no se reduce a ella.

XIII. PRINCIPIO DE MEMORIA POR REACTIVIDAD

La memoria funcional de CTNet se entiende mejor como una propiedad de entrada-salida del sistema completo. Sea $\mathcal{S}$ el sistema en un instante dado, y sea u un estímulo. La respuesta observable se escribe

$$y = F_{\mathcal{S}}(u). \quad (54)$$

Una experiencia e no necesita quedar almacenada como registro explícito. Basta con que transforme el operador de respuesta:

$$F_{\mathcal{S}} \longmapsto F_{\mathcal{S}|e}. \quad (55)$$

La memoria producida por e se mide por la diferencia funcional inducida sobre una clase de estímulos:

$$\Delta_e(\mathcal{U}) = \int_{\mathcal{U}} d(F_{\mathcal{S}|e}(u), F_{\mathcal{S}}(u)) \, d\mu(u). \quad (56)$$

Si $\Delta_e(\mathcal{U}) > 0$, entonces la experiencia está funcionalmente presente aunque no exista una copia directa del dato original.

Este punto separa dos nociones que suelen confundirse:

$$\text{memoria explícita} := \text{contenido materialmente guardado}, \quad (57)$$

$$\text{memoria funcional} := \text{cambio estable de respuesta ante estímulos}. \quad (58)$$

CTNet explota la segunda. Una secuencia de datos puede comprimirse en una modificación de la dinámica interna. Por ello, el tamaño físico del soporte no mide directamente la capacidad efectiva del sistema.

Una forma informacional de esta idea se obtiene comparando el coste de almacenar registros con el coste de almacenar una regla. Sea $D = \{(u_i, y_i)\}_{i=1}^n$ un conjunto de pares de entrenamiento. Un almacenamiento extensional guarda n pares. Un almacenamiento intensional guarda una descripción g tal que

$$g(u_i) = y_i, \quad i = 1, \dots, n. \quad (59)$$

La ganancia de compresión funcional puede escribirse como

$$G(D, g) = \frac{\sum_{i=1}^n \ell(u_i, y_i)}{\ell(g) + \ell(\epsilon_g)}, \quad (60)$$

con ℓ una longitud de descripción y ϵ_g el error residual de la regla. En el caso ideal $\epsilon_g = 0$, la memoria crece en número de casos cubiertos mientras el peso explícito queda concentrado en la regla.

XIV. CAPACIDAD EFECTIVA SIN CRECIMIENTO DE SLOTS

Sea B el número de registros físicos explícitos. En una memoria de slots convencional, la capacidad primaria escala con B . En CTNet, B no agota la capacidad porque los datos pueden afectar a tres niveles simultáneos:

1. estado latente persistente;
2. parámetros o módulos entrenables;
3. reglas procedimentales direccionadas por clave.

La memoria efectiva puede formalizarse como el rango funcional de la familia de respuestas

$$\mathfrak{F} = \{F_{\Omega, \theta, \rho} : \Omega \in \mathcal{O}, \theta \in \Theta, \rho \in \mathcal{P}\}, \quad (61)$$

siendo Ω el estado persistente, θ los parámetros y ρ las reglas procedimentales. La capacidad no queda determinada por $\dim(\Omega)$, sino por la cardinalidad efectiva de funciones distinguibles en $\mathfrak{F}$.

Para una tolerancia $\varepsilon > 0$ y una distribución de estímulos μ , definimos

$$N_\varepsilon(\mathfrak{F}) = \text{máx} \{m : \exists F_1, \dots, F_m \in \mathfrak{F}, \|F_i - F_j\|_{L^2(\mu)} > \varepsilon \forall i \neq j\}. \quad (62)$$

La capacidad funcional de resolución se mide entonces por

$$C_\varepsilon(\mathfrak{F}) = \log N_\varepsilon(\mathfrak{F}). \quad (63)$$

Este valor puede aumentar por reorganización dinámica y reglas recuperables sin aumentar el número de slots materiales.

XV. ARQUITECTURA COMPLETA DE PROCESAMIENTO

La versión operacional de CTNet puede expresarse como un circuito de ocho fases:

$$o_t \xrightarrow{\text{Enc}} S_t, \quad (64)$$

$$S_t \xrightarrow{\text{pack}} \Xi_t, \quad (65)$$

$$\Xi_t \xrightarrow{\Phi_\theta} \Xi_{t+1}, \quad (66)$$

$$\Xi_{t+1} \xrightarrow{\text{unpack}} (Z_{t+1}, M_{t+1}, R_{t+1}, C_{t+1}, P_{t+1}), \quad (67)$$

$$(Z, M, R)_{t+1} \xrightarrow{Q_\rho} q_{t+1}, \quad (68)$$

$$\Xi_{t+1} \xrightarrow{A_\eta} \hat{y}_t, \quad (69)$$

$$\hat{y}_t \xrightarrow{\text{Enc}} \Xi_{t+1}^{\text{out}}, \quad (70)$$

$$(\Xi_t, \Xi_{t+1}, \Xi_{t+1}^{\text{out}}) \xrightarrow{\mathcal{L}} \text{actualización}. \quad (71)$$

Cada fase tiene una función convencional: codificar, empaquetar, transformar, descomponer, diagnosticar, actuar, reobservar y entrenar.

Una forma compacta del paso completo es

$$\Xi_{t+1} = \Phi_\theta (\Xi_t + \text{Enc}_\psi(o_t) + \Gamma(q_t)), \quad (72)$$

con Γ una inserción diferenciable de diagnósticos internos. La salida es

$$\hat{y}_t = A_\eta(\Xi_{t+1}), \quad (73)$$

pero el sistema no considera esta salida como terminal: la salida se convierte en nuevo estímulo de evaluación.

XVI. AUTOCOMPRESIÓN Y DISMINUCIÓN DEL PESO EXPLÍCITO

CTNet permite que el peso físico de información explícita disminuya cuando una regularidad queda absorbida en la dinámica. Sea D_t el conjunto de registros explícitos disponibles y sea G_t el conjunto de reglas, rutas o transformaciones internas que reproducen sus efectos. Una política de autocompresión busca

$$\min_{G_t} \ell(G_t) + \lambda \sum_{(u,y) \in D_t} d(G_t(u), y) \quad (74)$$

conservando la respuesta funcional. Si existe G_t con error cero o absorbible, los datos individuales pueden dejar de ser necesarios como registros pesados.

La cantidad de información explícita almacenada se puede medir como

$$W_{\text{phys}}(t) = \sum_{r \in D_t} \ell(r) + \sum_{g \in G_t} \ell(g), \quad (75)$$

pero la capacidad funcional depende de

$$W_{\text{func}}(t) = \log |\{u \mapsto F_t(u) : u \in \mathcal{U}\}|. \quad (76)$$

La propiedad deseada es

$$W_{\text{phys}}(t+1) < W_{\text{phys}}(t), \quad W_{\text{func}}(t+1) > W_{\text{func}}(t), \quad (77)$$

lo que expresa aumento de capacidad por compresión reactiva.

XVII. RESOLUCIÓN VERIFICABLE COMO OPERADORES DE DOMINIO

Cuando CTNet se usa para resolver problemas, la salida libre no es suficiente. Una capa convencional de resolución introduce operadores de dominio. Cada operador consta de cuatro componentes:

$$\mathcal{D}_j = (P_j, R_j, U_j, V_j), \quad (78)$$

con:

- P_j : detector de aplicabilidad del dominio;
- R_j : transformación que reduce el problema a invariantes relevantes;

- U_j : generador de salida positiva mínima;
- V_j : verificador externo de la salida.

Dado un problema x , el router elige

$$j^* = \arg \max_j P_j(x). \quad (79)$$

La respuesta del operador es

$$\hat{y} = U_{j^*}(R_{j^*}(x)), \quad (80)$$

pero el cierre solo se acepta si

$$V_{j^*}(x, \hat{y}) = 1. \quad (81)$$

Si ningún operador produce una salida verificable, el sistema devuelve necesidad de nuevo operador de dominio en lugar de fingir resolución.

Esta capa traduce a lenguaje convencional la idea de que muchas familias de problemas se vuelven simples cuando se proyectan a los invariantes adecuados. No se trata de generar una respuesta plausible; se trata de transformar el problema a una forma donde la salida queda determinada y pueda verificarse.

XVIII. COMPILACIÓN AUTOMÁTICA DE OPERADORES VERIFICABLES

El siguiente paso natural es compilar nuevos operadores de dominio a partir de ejemplos o especificaciones. Sea

$$E = \{(x_i, y_i)\}_{i=1}^n \quad (82)$$

una colección de ejemplos. El compilador busca una regla g en una familia segura $\mathcal{G}$:

$$g^* = \arg \min_{g \in \mathcal{G}} \sum_{i=1}^n d(g(x_i), y_i) + \lambda \text{ complexity}(g). \quad (83)$$

Una vez hallada, no se incorpora directamente al sistema. Primero se construye un verificador V_g y se exige

$$V_g(x_i, g(x_i)) = 1, \quad i = 1, \dots, n. \quad (84)$$

Después se ejecutan casos adversariales A_g diseñados para romper la regla:

$$\forall a \in A_g, \quad V_g(a, g(a)) = 1 \quad \text{o} \quad g(a) = \perp. \quad (85)$$

El símbolo $\perp$ significa rechazo explícito: el operador reconoce que no aplica. Esta condición evita falsos positivos.

Los operadores compilados no necesitan ser redes neuronales. Pueden ser reglas simbólicas, programas declarativos, transformaciones sobre listas, evaluadores aritméticos, verificadores lógicos, algoritmos de grafos o procedimientos de optimización. CTNet actúa como sistema de estado, memoria funcional, selección y autoobservación; los operadores de dominio aportan cierre verificable cuando la tarea es formal.

XIX. ALGORITMOS DE ENTRENAMIENTO Y USO

El entrenamiento general puede escribirse como:

1. observar una entrada externa o interna o_t ;
2. codificarla como estado Ξ_t ;
3. aplicar una o varias transiciones reversibles;
4. calcular coherencia multivista, error no absorbido y estabilidad;
5. producir una acción visible $\hat{y}_t$;
6. reobservar $\hat{y}_t$;
7. aplicar verificación externa si existe;
8. actualizar parámetros, estado persistente o reglas procedimentales.

La pérdida total ampliada puede tomar la forma

$$\begin{aligned} \mathcal{L}_{\text{full}} = & \mathcal{L}_{\text{task}} + \lambda_1 \mathcal{L}_{\text{up}} + \lambda_2 \mathcal{L}_{\text{coh}} + \lambda_3 \mathbb{E}[\omega] - \lambda_4 \mathbb{E}[s] \\ & + \lambda_5 \mathcal{L}_{\text{rev}} + \lambda_6 \mathcal{L}_{\text{out_reentry}} + \lambda_7 \mathcal{L}_{\text{proc}} + \lambda_8 \mathcal{L}_{\text{verify}}. \end{aligned} \quad (86)$$

El término de verificación puede definirse como

$$\mathcal{L}_{\text{verify}} = \begin{cases} 0, & V(x, \hat{y}) = 1, \\ c_{\text{fail}}, & V(x, \hat{y}) = 0, \\ c_{\text{none}}, & \text{no existe verificador aplicable.} \end{cases} \quad (87)$$

Cuando no existe verificador, el sistema no fuerza una conclusión: produce una señal de nuevo dominio a cubrir.

XX. INTERPRETACIÓN CONVENCIONAL DE LA MEJORA ACUMULATIVA

La mejora acumulativa de CTNet no consiste solo en reducir una pérdida sobre un conjunto fijo. Consiste en expandir el conjunto de transformaciones verificables disponibles. Sea $\mathcal{D}_t$ el conjunto de operadores de dominio en el instante t . La cobertura verificable es

$$\text{Cov}(\mathcal{D}_t) = \mu(\{x \in \mathcal{X} : \exists j \in \mathcal{D}_t, V_j(x, U_j(R_j(x))) = 1\}). \quad (88)$$

Añadir un operador correcto aumenta cobertura:

$$\text{Cov}(\mathcal{D}_{t+1}) \geq \text{Cov}(\mathcal{D}_t). \quad (89)$$

Si además el operador reemplaza muchos casos almacenados por una regla compacta, disminuye peso explícito y aumenta capacidad efectiva.

Este mecanismo convierte los fallos en información estructural. Un fallo verificable indica que el problema pertenece a una zona sin operador adecuado, o que el operador actual no captura el invariante suficiente. El sistema puede entonces generar ejemplos, buscar una regla, construir un verificador y ampliar el atlas de operadores.

XXI. EQUIVALENCIAS CON CONCEPTOS ESTÁNDAR

La arquitectura puede expresarse sin terminología nueva mediante la siguiente tabla conceptual:

Estado empaquetado:

Espacio latente compuesto con subespacios funcionales.

Memoria reactiva:

Memoria implícita codificada en la función de transición.

Métricas internas:

Diagnóstico diferenciable de estabilidad e inconsistencia.

Consistencia multivista:

Regularización local-global entre proyecciones del estado.

Transformación reversible:

Dinámica de acoplamiento invertible para preservar información.

Reentrada de salida:

Aprendizaje en bucle cerrado por acción y reobservación.

Memoria procedimental:

Reglas y rutas recuperables por clave, no almacenamiento plano.

Operadores de dominio:

Solvers/verificadores específicos invocados por aplicabilidad.

Compilador de operadores:

Inducción de reglas verificables a partir de ejemplos.

XXII. CORRESPONDENCIA DIRECTA CON LA IMPLEMENTACIÓN

La formulación anterior no es una reconstrucción abstracta separada del código. Cada bloque matemático corresponde a una pieza concreta de la implementación. La clase `Observador` contiene los campos

$$o = (x, y, \text{source}, \text{regime}), \quad (90)$$

que representan contenido, objetivo local, procedencia y tipo de contexto. La función `batch_to_state` implementa el codificador

$$\text{Enc}_\psi : o \mapsto (Z, M, R, C, P), \quad (91)$$

con cuatro rutas textuales diferenciadas: contenido principal, memoria, relaciones y objetivo. En notación convencional:

$$Z = E_Z(\text{regime}, x), \quad (92)$$

$$M = E_M(\text{source}, \text{regime}, x), \quad (93)$$

$$R = E_R(\text{source}, \text{regime}, x), \quad (94)$$

$$Y = E_Y(y). \quad (95)$$

Las amplitudes de memoria y relaciones se introducen con menor escala inicial que la representación activa. Esta decisión separa señal principal de contexto reactivo: el contenido de trabajo entra con amplitud plena, mientras que memoria y relaciones actúan como moduladores de trayectoria.

El empaquetado implementado por `FoldLayout.pack` realiza una concatenación estructurada:

$$\Xi = \text{reshape}([\text{vec}(Z), \text{vec}(M), \text{vec}(R), C, P]). \quad (96)$$

La operación inversa `unpack` recupera exactamente las mismas regiones. Por tanto, la arquitectura no depende de interpretar todo el estado como una matriz homogénea sin papel interno. El tensor global es homogéneo para la dinámica, pero heterogéneo para la interpretación funcional.

XXIII. CODIFICACIÓN DETERMINISTA DE SEÑALES TEXTUALES

El sistema usa una codificación determinista de bytes. Para un texto x , truncado a $B_{\text{máx}}$ bytes, se define una señal $v \in \mathbb{R}^n$ mediante acumulación circular:

$$v_j = \sum_{i: i \bmod n = j} \frac{b_i/127.5 - 1}{\sqrt{1 + \lfloor i/n \rfloor}}, \quad (97)$$

seguida de una perturbación sinusoidal fija y saturación suave:

$$\tilde{v}_j = \tanh \left(v_j + 0.015 \sin \frac{2\pi j}{n} + 0.0075 \cos \frac{4\pi j}{n} \right). \quad (98)$$

Esta codificación no es una tokenización semántica pesada. Es una proyección determinista que convierte texto en perturbación inicial del estado. La semántica no se supone completa en el codificador; se produce por la dinámica posterior, por la memoria funcional y por la coherencia entre vistas.

Esta elección tiene una consecuencia importante: la entrada textual no se almacena como secuencia a copiar, sino como campo inicial que deforma el estado. Dos textos pueden compartir bytes, fuentes o regímenes y producir trayectorias cercanas; dos textos formalmente parecidos pueden separarse si inducen relaciones distintas en M y R .

XXIV. DESCOMPOSICIÓN DE CAPACIDAD TENSORIAL

En la configuración base del código, el estado plegado tiene forma

$$\Xi \in \mathbb{R}^{B \times N \times d}, \quad N = 64, \quad d = 16, \quad (99)$$

con capacidad física

$$D = Nd = 1024. \quad (100)$$

La distribución funcional por defecto es

$$Z \in \mathbb{R}^{B \times 32 \times 16}, \quad (101)$$

$$M \in \mathbb{R}^{B \times 8 \times 16}, \quad (102)$$

$$R \in \mathbb{R}^{B \times 8 \times 16}, \quad (103)$$

$$C \in \mathbb{R}^{B \times 29}, \quad (104)$$

$$P \in \mathbb{R}^{B \times 227}. \quad (105)$$

La suma coincide con 1024. La reserva P no es residuo sin uso: es margen estructural. Permite que el estado completo sea transformado sin forzar a las regiones semánticas a ocupar toda la capacidad física. En lenguaje convencional, P funciona como espacio de maniobra latente.

La memoria efectiva no se mide por $\dim(M)$, sino por la clase de funciones de respuesta que el sistema puede implementar. Sea

$$F_t : x \mapsto \text{Dec}(\Phi_{\theta_t}(\text{Enc}(x))) \quad (106)$$

la función inducida por el sistema en el instante t . La memoria funcional contenida entre dos instantes puede modelarse como variación de función:

$$\Delta F_t = F_{t+1} - F_t. \quad (107)$$

Aunque el soporte físico sea acotado, el conjunto de respuestas distinguibles puede aumentar si la actualización produce nuevas separaciones funcionales. Una medida convencional de capacidad efectiva es el número de respuestas separables por una tolerancia ε :

$$C_{\text{eff}}(t, \varepsilon) = \log N_{\varepsilon}(\{F_t(x) : x \in \mathcal{X}\}), \quad (108)$$

donde N_{ε} es el número de bolas necesarias para cubrir la imagen de respuestas. Esta magnitud puede aumentar aunque D permanezca constante.

XXV. MEMORIA COMO REACTIVIDAD: JACOBIANOS Y NÚCLEOS DE RESPUESTA

La forma convencional más precisa de describir la memoria de CTNet es mediante reactividad. Para una entrada x , definimos el Jacobiano de respuesta

$$J_t(x) = \frac{\partial F_t(x)}{\partial x}. \quad (109)$$

La memoria adquirida por experiencia no se limita a un registro r_i almacenado. Se expresa como cambio en el campo de Jacobianos:

$$\mathcal{M}_{t \rightarrow t+1}(x) = J_{t+1}(x) - J_t(x). \quad (110)$$

Dos estímulos x y x' quedan funcionalmente relacionados si sus respuestas diferenciales quedan alineadas. Esto define un núcleo de reactividad:

$$K_t(x, x') = \langle J_t(x), J_t(x') \rangle_F. \quad (111)$$

La memoria funcional aumenta cuando K_t adquiere estructura discriminativa: ciertos estímulos empiezan a activar rutas parecidas, otros se separan, y otros disparan correcciones específicas.

Esta formulación explica por qué el peso físico de la información explícita puede disminuir mientras aumenta la capacidad. Si muchos ejemplos $\{x_i, y_i\}_{i=1}^m$ son reemplazados por una transformación que reproduce sus efectos, el almacenamiento extensional baja:

$$W_{\text{explicit}} = \sum_{i=1}^m |x_i| + |y_i|, \quad (112)$$

pero la competencia funcional se conserva o aumenta si

$$F_t(x_i) = y_i \quad \text{para los casos vistos}, \quad (113)$$

y además se extiende a entradas no vistas de la misma familia. La memoria no está en la copia, sino en la reacción.

XXVI. MÉTRICA DE ESTABILIDAD DE SEIS VARIABLES

El componente diagnóstico de seis variables calcula una firma de estabilidad a partir de la representación activa, la memoria y las relaciones. En términos convencionales, se computan

seis rasgos:

$$a_1 = \left(\frac{1}{BTd} \sum z^2 \right)^{1/2}, \quad (114)$$

$$a_2 = \left(\frac{1}{BTd/2} \sum (u - p)^2 \right)^{1/2}, \quad (115)$$

$$a_3 = \left(\frac{1}{Bsd} \sum M^2 \right)^{1/2}, \quad (116)$$

$$a_4 = \left(\frac{1}{Bed} \sum R^2 \right)^{1/2}, \quad (117)$$

$$a_5 = |\cos(\bar{z}, \bar{M} + \bar{R})|, \quad (118)$$

$$a_6 = \frac{1}{1 + \sigma(a_2 + 0.25a_3 + 0.25a_4 - a_5)}. \quad (119)$$

Estos rasgos representan magnitud activa, separación interna, presencia de memoria, intensidad relacional, alineación contextual y estabilidad preliminar.

El sistema transforma esos rasgos en quince ángulos, uno por cada plano de un espacio de seis dimensiones:

$$\theta_{ij} = \pi \tanh(b_{ij} + s_{ij}(a_i - a_j)), \quad 1 \leq i < j \leq 6. \quad (120)$$

Se aplican rotaciones planas a un conjunto fijo de estados discretos y se selecciona el estado con menor distancia a la firma observada. La distancia residual mínima es

$$r = \min_k \|\rho_k(\theta) - (a - 1/2)\|_2. \quad (121)$$

La absorción interna se calcula como una combinación diferenciable de cierre, baja separación, mediación y estabilidad:

$$A = \alpha (0.22 + 0.48c + 0.15(1 - a_2) + 0.10a_5 + 0.05a_6)_{[0,1]}. \quad (122)$$

La inconsistencia no absorbida y la puntuación de estabilidad quedan

$$\omega = \max(0, r - A), \quad s = \exp(-\omega). \quad (123)$$

Esto es una medición interna de estabilidad, no una etiqueta externa. Su función es guiar la dinámica y señalar cuánta tensión estructural queda sin integrar.

XXVII. TRANSFORMACIÓN REVERSIBLE Y PRESERVACIÓN DE INFORMACIÓN

El núcleo dinámico usa bloques de acoplamiento aditivo. Dividimos el tensor por la última dimensión:

$$x = (x_1, x_2). \quad (124)$$

El bloque directo es

$$y_1 = x_1 + \gamma f(x_2), \quad (125)$$

$$y_2 = x_2 + \gamma g(y_1). \quad (126)$$

La inversa exacta es

$$x_2 = y_2 - \gamma g(y_1), \quad (127)$$

$$x_1 = y_1 - \gamma f(x_2). \quad (128)$$

La propiedad central es que f y g no necesitan ser invertibles para que el bloque completo lo sea. La invertibilidad procede de la triangularidad del acoplamiento. Esta clase de transformaciones permite profundidad sin destrucción sistemática de coordenadas internas.

La versión fractal añade un procesador latente auxiliar sobre la mitad del estado:

$$u' = u + \gamma_\ell L(p), \quad (129)$$

$$y_1 = u' + \gamma_m f(p), \quad (130)$$

$$y_2 = p + \gamma_m g(y_1), \quad (131)$$

con inversa construida en orden inverso. En lenguaje convencional, esto es una composición reversible con subdinámica compartida. El efecto funcional es permitir que una parte del estado module a otra sin perder la capacidad de reconstruir la entrada.

XXVIII. OPERADORES HIPERRADIALES COMO MEZCLA LOCAL DE CONTEXTO

Los operadores internos f y g se implementan mediante kernels de desplazamiento en varios radios. Para radios $\mathcal{R} = \{1, 2, 4\}$,

$$H(x)_i = \lambda \phi \left(\text{norm} \left(\sum_{r \in \mathcal{R}} w_{r,-} x_{i-r} + w_{r,0} x_i + w_{r,+} x_{i+r} \right) \right), \quad (132)$$

donde los pesos $w_{r,*}$ se normalizan por *softmax*, ϕ es una activación tipo GELU y la normalización es RMSNorm. Este operador mezcla contexto local y desplazado sin usar atención completa. Su papel convencional es parecido a una convolución circular multi-rango con pesos adaptativos y activación estabilizada.

Como el operador actúa en la dimensión de tokens latentes, no requiere que la secuencia original siga viva como tokens textuales. La entrada ya fue transformada en estado; el kernel opera sobre coordenadas de estado, no sobre palabras.

XXIX. COHERENCIA DIAGONAL Y DE BAJO RANGO

La energía de coherencia combina una discrepancia de partición con una métrica aprendida. Si u y p son dos mitades de la última dimensión, el sistema estima

$$\hat{u} = L(p), \quad \hat{p} = L^{-1}(u), \quad (133)$$

y penaliza

$$E_0 = \|u - \hat{u}\|^2 + \|p - \hat{p}\|^2. \quad (134)$$

Sobre el estado centrado $\tilde{x} = x - \bar{x}$, se calcula una información de segundo orden con parte diagonal y parte de bajo rango:

$$I = \sum_j d_j \text{Var}(x_j) + \sum_{r=1}^q \mathbb{E} [(\tilde{x}^\top \ell_r)^2]. \quad (135)$$

La energía final se escala por

$$v = \exp(\beta \text{clip}(I/d, -5, 5)), \quad E_{\text{coh}} = vE_0. \quad (136)$$

Esta construcción equivale a una métrica de coherencia adaptativa: el sistema no mide todos los errores con el mismo peso, sino que aprende qué direcciones del estado importan más para la estabilidad funcional.

XXX. CONSISTENCIA MULTIESCALA LOCAL-GLOBAL

La pérdida local-global se calcula sobre múltiples vistas del estado. Sea h un tensor cualquiera entre $Z, M, R, C, \Xi, \Delta\Xi$. Se divide su última dimensión en dos mitades:

$$h = (h_u, h_p), \quad (137)$$

y se define

$$\mathcal{L}_{up}(h) = \|h_u - h_p\|^2. \quad (138)$$

La implementación no aplica esto solo una vez. También evalúa desplazamientos internos y agrupaciones por escala:

$$\mathcal{L}_{ms}(h) = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \mathcal{L}_{up}(v(h)), \quad (139)$$

donde $\mathcal{V}$ contiene identidad, desplazamientos en dimensión de rasgos, desplazamientos en dimensión de tokens y promedios por escalas 2, 4, 8.

La pérdida total de consistencia expuesta es

$$\mathcal{L}_{all} = \frac{1}{6} (\mathcal{L}_{ms}(Z) + \mathcal{L}_{ms}(M) + \mathcal{L}_{ms}(R) + \mathcal{L}_{ms}(C) + \mathcal{L}_{ms}(\Xi) + \mathcal{L}_{ms}(\Delta\Xi)). \quad (140)$$

En términos convencionales, esto es una regularización de consistencia multivista y multi-escala. Fuerza que partes, agregados y deformaciones del estado conserven compatibilidad interna.

XXXI. AUTOOBSERVACIÓN COMO GENERACIÓN DE DATOS INTERNOS

CTNet convierte sus transiciones internas en nuevas observaciones entrenables. Dado un estado antes Ξ_t , un estado después Ξ_{t+1} y su diferencia

$$\Delta\Xi_t = \Xi_{t+1} - \Xi_t, \quad (141)$$

se construye una señal de proceso

$$q_t = [\Xi_t, \Xi_{t+1}, \Delta\Xi_t, C_t, C_{t+1}, C_{t+1} - C_t]. \quad (142)$$

Esta señal se pliega de forma determinista a la misma estructura que una entrada externa:

$$q_t \mapsto (Z^{\text{proc}}, M^{\text{proc}}, R^{\text{proc}}, C^{\text{proc}}, P^{\text{proc}}). \quad (143)$$

El resultado no es un log externo. Es un nuevo ejemplo de entrenamiento que atraviesa la misma red. En lenguaje convencional, esto es aprendizaje auto-supervisado sobre trazas internas del proceso.

La pérdida asociada combina consistencia multivista, energía de coherencia, inconsistencia no absorbida y puntuación de estabilidad:

$$\mathcal{L}_{\text{proc}} = \mathcal{L}_{up}^{\text{proc}} + \lambda_c E_{\text{coh}}^{\text{proc}} + \lambda_\omega \mathbb{E}[\omega^{\text{proc}}] - \lambda_s \mathbb{E}[s^{\text{proc}}]. \quad (144)$$

Esto permite que el sistema aprenda no solo de los datos que recibe, sino de la forma en que cambia al procesarlos.

XXXII. SALIDA VISIBLE COMO ACCIÓN Y NO COMO CENTRO DE LA ARQUITECTURA

La salida textual se obtiene proyectando el estado transformado a una secuencia de caracteres visibles. Si z es la región activa del estado final, se aplica

$$a_i = \left\lfloor 32 + 94 \frac{\tanh(z_i) + 1}{2} \right\rfloor, \quad (145)$$

con recorte al rango ASCII imprimible. Esta salida no pretende ser una descripción completa del estado. Es una acción visible generada por el estado.

La diferencia con un decodificador autoregresivo es estructural. Un decoder autoregresivo modela

$$p(y_i \mid y_{<i}, x). \quad (146)$$

El efector de CTNet implementa

$$y = G(\Xi_T), \quad (147)$$

y después reingiere y como observación:

$$\Xi_y = \text{Enc}(y), \quad \mathcal{L}_{\text{out}} = \mathcal{L}(\Phi_\theta(\Xi_y)). \quad (148)$$

La salida no queda fuera del sistema. Vuelve a entrar y debe sostener coherencia.

XXXIII. MEMORIA PROCEDIMENTAL DIRECCIONABLE POR CLAVE

El módulo procedimental implementa memoria por coordenadas formulaicas. Dada una clave k , se calcula una coordenada mediante funciones hash y generadores deterministas:

$$c(k) = H(\text{secret}, k). \quad (149)$$

El dato puede guardarse como cápsula reversible o como regla. En modo cápsula, se aplica una transformación Feistel:

$$z = \mathcal{F}_{\kappa(k)}(d), \quad d = \mathcal{F}_{\kappa(k)}^{-1}(z), \quad (150)$$

con autenticación HMAC. En modo regla, el recibo no contiene necesariamente todo el dato; contiene una descripción procedimental evaluable:

$$d = \text{Eval}(k, \rho). \quad (151)$$

Esto es memoria intensional: se guarda una ruta de reconstrucción o generación, no siempre una copia pesada.

La operación `fold` actualiza un resumen interno Ω por XOR con palabras derivadas del recibo:

$$\Omega_{t+1} = \Omega_t \oplus h(\text{receipt}). \quad (152)$$

Aplicar el mismo pliegue de nuevo revierte el resumen:

$$\Omega_{t+2} = \Omega_t. \quad (153)$$

Por eso el módulo puede auditar reversibilidad del pliegue y lectura directa. En lenguaje convencional: es una memoria procedimental autenticada con coordenadas virtuales no asignadas por slots materiales crecientes.

XXXIV. ENTRENAMIENTO COMPLETO EXPRESADO COMO ALGORITMO

La iteración de entrenamiento puede escribirse como:

1. Tomar un lote de observaciones $\{o_i\}_{i=1}^B$.
2. Codificarlo como estado $S_t = \text{Enc}(o_{1:B})$.
3. Calcular $S_{t+1} = \Phi_\theta(S_t)$.
4. Empaquetar Ξ_{t+1} y medir coherencia E_{coh} .
5. Medir consistencia multivista $\mathcal{L}_{\text{all}}$.
6. Calcular diagnóstico interno (r, A, ω, s) .
7. Si corresponde, generar observación interna del proceso y su pérdida.
8. Si corresponde, generar salida visible, reingresarla y calcular pérdida de reentrada.
9. Medir reversibilidad con Φ_θ^{-1} en pasos seleccionados.

10. Actualizar parámetros por AdamW con recorte de gradiente.

La pérdida implementada responde a la forma

$$\begin{aligned} \mathcal{L} = & \lambda_{up} \mathcal{L}_{\text{all}} + \lambda_a \|Z_{out} - Y\|^2 + \lambda_c E_{\text{coh}} + \lambda_\omega \mathbb{E}[\omega] + \lambda_q \|C_{out} - Q(Z, M, R)\|^2 \\ & + \lambda_s \mathcal{L}_{\text{structure}} + \lambda_r \mathcal{L}_{\text{rev}} + \lambda_p \mathcal{L}_{\text{proc}} + \lambda_e \mathcal{L}_{\text{efector}}. \end{aligned} \quad (154)$$

El término de estructura conserva varianza mínima en memoria y relaciones:

$$\mathcal{L}_{\text{structure}} = \text{máx}(0, \sigma_0^2 - \text{Var}(M)) + \text{máx}(0, \sigma_0^2 - \text{Var}(R)). \quad (155)$$

Esta forma evita que las regiones funcionales colapsen a constantes inertes.

XXXV. CAPACIDAD CRECIENTE CON PESO EXPLÍCITO DECRECIENTE

Sea D_t el conjunto de registros explícitos conservados y sea Θ_t la configuración funcional del sistema. Definimos el peso físico explícito como

$$W_t = \sum_{d \in D_t} \text{bytes}(d). \quad (156)$$

Definimos la competencia funcional sobre una familia $\mathcal{F}$ como

$$A_t(\mathcal{F}) = \Pr_{(x,y) \sim \mathcal{F}} [V(x, F_t(x), y) = 1]. \quad (157)$$

La tesis operacional de memoria por reactividad se expresa como la posibilidad de trayectorias donde

$$W_{t+1} < W_t, \quad (158)$$

y al mismo tiempo

$$A_{t+1}(\mathcal{F}) \geq A_t(\mathcal{F}), \quad C_{\text{eff}}(t+1, \varepsilon) > C_{\text{eff}}(t, \varepsilon). \quad (159)$$

Esto ocurre cuando ejemplos almacenados como casos son sustituidos por una regla, una transformación de estado o un cambio estable de reactividad. La memoria mejora no por acumulación de copias, sino por conversión de casos en forma funcional.

XXXVI. RESOLUCIÓN MEDIANTE OPERADORES CONVENCIONALES

La capa de resolución verificable puede describirse sin terminología especial. Sea $\mathcal{O} = \{O_j\}$ un conjunto de operadores de dominio. Cada operador contiene cuatro funciones:

$$O_j = (D_j, R_j, U_j, V_j), \quad (160)$$

donde D_j decide aplicabilidad, R_j reduce la entrada a una representación interna, U_j produce una salida observable y V_j verifica la respuesta. La resolución es

$$\hat{y} = U_j(R_j(x)), \quad \text{solved} = V_j(x, \hat{y}). \quad (161)$$

La arquitectura completa no declara una respuesta cerrada por coherencia interna solamente. Cuando existe un verificador de dominio, el cierre operativo exige

$$V_j(x, \hat{y}) = 1. \quad (162)$$

El compilador de operadores toma ejemplos

$$\mathcal{E} = \{(x_i, y_i)\}_{i=1}^m \quad (163)$$

e intenta inferir una regla ρ tal que

$$V_\rho(x_i, U_\rho(R_\rho(x_i))) = 1 \quad \forall i. \quad (164)$$

Cuando la regla queda validada, se guarda como operador reutilizable. En lenguaje convencional, esto es inducción de programa declarativo con verificación externa.

XXXVII. RELACIÓN ENTRE ESTADO CONTINUO Y OPERADORES DISCRETOS

CTNet contiene dos niveles compatibles. El primero es continuo y latente:

$$\Xi_{t+1} = \Phi_\theta(\Xi_t). \quad (165)$$

El segundo es discreto y verificable:

$$x \xrightarrow{O_j} \hat{y} \xrightarrow{V_j} \{0, 1\}. \quad (166)$$

La relación convencional entre ambos niveles es la de un sistema híbrido. El estado continuo mantiene contexto, historial, reactividad, autodiagnóstico y selección; los operadores discretos ejecutan cierres exactos cuando el dominio está tipado.

La selección de operador puede modelarse como

$$j^*(x) = \arg \max_j \pi_j(x, \Xi_t), \quad (167)$$

donde π_j es una puntuación de aplicabilidad. La respuesta final puede depender de ambos niveles:

$$\hat{y} = \begin{cases} U_{j^*}(R_{j^*}(x)), & V_{j^*} = 1, \\ G(\Phi_\theta(\text{Enc}(x))), & \text{si no hay operador exacto aplicable.} \end{cases} \quad (168)$$

Esta arquitectura no opone red neuronal y cálculo simbólico. Los integra: la red mantiene reactividad y selección; el operador cierra familias formales con verificación.

XXXVIII. ACUMULACIÓN DE CONOCIMIENTO COMO CAMBIO DE FUNCIÓN

El aprendizaje acumulativo de CTNet puede representarse como una secuencia de funciones

$$F_0, F_1, F_2, \dots, F_t, \quad (169)$$

y una secuencia de operadores verificables

$$\mathcal{O}_0 \subseteq \mathcal{O}_1 \subseteq \dots \subseteq \mathcal{O}_t. \quad (170)$$

La memoria total efectiva combina ambas partes:

$$\mathfrak{M}_t = (F_t, \mathcal{O}_t, K_t), \quad (171)$$

donde K_t es el núcleo de reactividad. Un nuevo conocimiento queda incorporado si produce al menos una de estas tres transformaciones:

$$F_{t+1} \neq F_t, \quad (172)$$

$$\mathcal{O}_{t+1} \supset \mathcal{O}_t, \quad (173)$$

$$K_{t+1} \neq K_t. \quad (174)$$

Esta definición permite expresar memoria sin reducirla a almacenamiento físico. Recordar es modificar la función, ampliar operadores o cambiar relaciones de respuesta.

XXXIX. APRENDIZAJE POR CONTRASTE ENTRE COHERENCIA E INCOHERENCIA

El entrenamiento de CTNet no exige que toda trayectoria observada sea coherente. Una trayectoria incoherente también es informativa porque delimita una región de no cierre, induce una dirección de corrección y aumenta la perspectiva del sistema sobre el espacio de estados. En lenguaje convencional, la incoherencia actúa como señal negativa, ejemplo de frontera, contraejemplo operacional o gradiente de reorganización.

Sea Ξ_t un estado y sea $\tau = (\Xi_0, \dots, \Xi_T)$ una trayectoria inducida por observaciones externas o internas. Definimos una función de energía de consistencia

$$E_{\text{coh}}(\tau) = \sum_{k=1}^K \alpha_k d_k(\Pi_k(\Xi_T), \Pi'_k(\Xi_T)) + \beta \omega(\Xi_T), \quad (175)$$

donde las proyecciones Π_k, Π'_k comparan vistas locales y globales, y ω mide error estructural no integrado. Una trayectoria coherente tiene baja energía; una trayectoria incoherente tiene alta energía. Ambas actualizan la función de respuesta.

El aprendizaje puede expresarse como una actualización funcional

$$F_{t+1} = F_t + \Delta F(\tau, E_{\text{coh}}(\tau), y, V), \quad (176)$$

donde F_t es la función de transición efectiva del sistema, y es una salida o referencia cuando existe, y V un verificador opcional. No se exige que $E_{\text{coh}}(\tau)$ sea pequeño para que haya aprendizaje. Lo que se exige es que la trayectoria produzca una deformación funcional aprovechable:

$$\|F_{t+1} - F_t\|_{\mathcal{H}} > 0. \quad (177)$$

La incoherencia aporta conocimiento porque contiene información sobre la orientación del espacio de estados. Si $\mathcal{C}$ es el conjunto de estados cerrados y $\mathcal{I}$ el conjunto de estados no cerrados, entonces un ejemplo incoherente no es ruido plano; es una muestra de $\mathcal{I}$ que mejora la separación funcional entre regiones:

$$m(\Xi) = D(\Xi, \mathcal{I}) - D(\Xi, \mathcal{C}). \quad (178)$$

Aprender no consiste solo en atraer estados hacia $\mathcal{C}$, sino también en conocer cómo se organizan las regiones que no cierran. La frontera

$$\partial\mathcal{C} = \{\Xi : D(\Xi, \mathcal{C}) = D(\Xi, \mathcal{I})\} \quad (179)$$

es informativa porque determina qué perturbaciones mantienen cierre y cuáles lo rompen.

Esto permite distinguir tres tipos convencionales de muestra:

$$\tau^+ : E_{\text{coh}}(\tau) \leq \epsilon, \quad (180)$$

$$\tau^- : E_{\text{coh}}(\tau) > \epsilon, \quad (181)$$

$$\tau^\partial : |E_{\text{coh}}(\tau) - \epsilon| \leq \delta. \quad (182)$$

Las trayectorias τ^+ estabilizan patrones de cierre; las trayectorias τ^- enseñan direcciones de no cierre; las trayectorias de frontera τ^∂ enseñan sensibilidad, umbrales y condiciones de cambio de régimen.

Por tanto, la pérdida total no debe interpretarse como mandato de coherencia inmediata, sino como lectura de orientación:

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{task}} + \lambda_+ E_{\text{coh}}(\tau^+) + \lambda_- \rho(E_{\text{coh}}(\tau^-)) + \lambda_\partial |E_{\text{coh}}(\tau^\partial) - \epsilon|. \quad (183)$$

El término ρ no elimina la incoherencia; la convierte en señal contrastiva. Una elección simple es

$$\rho(E) = \log(1 + E), \quad (184)$$

que conserva información de magnitud sin permitir que un caso muy incoherente domine todo el entrenamiento. Otra opción es usar una pérdida de margen:

$$\rho(E) = \max(0, \mu - E), \quad (185)$$

cuando se desea que ciertos estados queden reconocidos como negativos claros.

La ganancia de perspectiva puede formalizarse como aumento de capacidad discriminativa del sistema. Sea $\mathcal{P}_t$ el conjunto de particiones funcionales que CTNet puede inducir sobre el espacio de estados en el tiempo t . Una trayectoria incoherente aporta perspectiva si

$$\mathcal{P}_{t+1} \succ \mathcal{P}_t, \quad (186)$$

es decir, si permite distinguir regiones que antes quedaban mezcladas. Equivalentemente, si κ_t es un núcleo de respuesta,

$$\kappa_{t+1}(x, x') \neq \kappa_t(x, x') \quad (187)$$

para pares de estímulos cuya diferencia era funcionalmente invisible antes del entrenamiento.

En esta lectura, un entrenamiento incoherente no degrada el sistema. Aumenta conocimiento si queda integrado como contraste, borde, dirección de rechazo, regla de no cierre o condición de reorientación. CTNet aprende de la coherencia porque estabiliza trayectorias que cierran, y aprende de la incoherencia porque registra cómo y dónde una trayectoria no cierra. Ambas son memoria funcional.

XL. FORMALIZACIÓN MATEMÁTICA DE u/p COMO GENERADOR DE OPERADORES

La formulación anterior muestra que CTNet aprende tanto de trayectorias coherentes como incoherentes. En esta sección se fija el mecanismo matemático que explica por qué esto ocurre: el principio u/p no actúa como una simple medida auxiliar de consistencia, sino como un generador de operadores de transformación. Una diferencia entre una parte y una totalidad relativa puede modificar el estado, la función de transición, la observabilidad interna, los criterios de aprendizaje, los roles funcionales y la clausura de composiciones disponibles.

A. Estado, transición y memoria como función de respuesta

Sea $\mathcal{X}$ el espacio de estímulos observables y sea $\mathcal{S}$ el espacio de estados internos. En cada instante t , CTNet posee un estado estructurado

$$s_t \in \mathcal{S}, \quad s_t = (Z_t, M_t, R_t, C_t, P_t), \quad (188)$$

donde Z_t representa activación latente, M_t memoria funcional, R_t relaciones internas, C_t cierre o estabilidad y P_t autoobservación propioceptiva. Un estímulo $x_t \in \mathcal{X}$ induce la transición

$$s_{t+1} = T_{\theta_t}(s_t, x_t). \quad (189)$$

El conocimiento efectivo no queda identificado con una lista de datos observados, sino con la transformación adquirida por la función de transición:

$$K_t \equiv T_{\theta_t}. \quad (190)$$

Por tanto, aprender no significa solamente construir

$$D_t = \{x_1, \dots, x_t\}, \quad (191)$$

sino modificar el modo en que futuros estímulos deforman el estado:

$$x \longmapsto T_{\theta_t}(s, x). \quad (192)$$

La memoria funcional inducida por el historial puede escribirse como

$$\mathcal{M}_t(x; s) = T_{\theta_t}(s, x). \quad (193)$$

Dos sistemas con la misma memoria explícita pueden tener memorias funcionales distintas si sus funciones de transición difieren:

$$M_t^{\text{exp}} = M_{t'}^{\text{exp}} \quad \text{pero} \quad T_{\theta_t} \neq T_{\theta_{t'}}. \quad (194)$$

La memoria real se reconoce entonces por la diferencia persistente de respuesta:

$$\Delta \mathcal{M}_{t,t'}(x; s) = T_{\theta_t}(s, x) - T_{\theta_{t'}}(s, x). \quad (195)$$

B. Residuo local-global inducido por u/p

Sea $\mathcal{P}_t = \{p_i^t\}_{i \in I_t}$ una familia de proyecciones internas

$$p_i^t : \mathcal{S} \longrightarrow \mathcal{S}_i, \quad (196)$$

y sea

$$u_i^t : \mathcal{S}_i \longrightarrow \mathcal{S} \quad (197)$$

un operador de elevación, reconstrucción o integración de la parte en una totalidad relativa.

El desacoplamiento u/p se mide mediante el residuo

$$\delta_i^t(s) = s - u_i^t p_i^t(s). \quad (198)$$

Si

$$\delta_i^t(s) = 0, \quad (199)$$

la parte $p_i^t(s)$ es compatible con la totalidad reconstruida. Si

$$\delta_i^t(s) \neq 0, \quad (200)$$

aparece una diferencia local-global. Esta diferencia no es ruido: es información direccional sobre la insuficiencia de la reconstrucción actual.

Definimos la energía local-global asociada a la vista i como

$$E_i^t(s) = \frac{1}{2} \|s - u_i^t p_i^t(s)\|_{G_t}^2, \quad (201)$$

donde G_t es una métrica interna dependiente del estado. El residuo total es

$$\Omega_t(s) = \sum_{i \in I_t} w_i^t E_i^t(s), \quad (202)$$

y el cierre global puede escribirse como

$$C_t(s) = \exp[-\Omega_t(s)]. \quad (203)$$

La coherencia corresponde a baja energía,

$$\Omega_t(s) \approx 0, \quad (204)$$

mientras que la incoherencia corresponde a energía positiva,

$$\Omega_t(s) > 0. \quad (205)$$

C. De residuo a campo vectorial y operador

La clave formal es que toda diferencia genera un campo de transformación. Dado el residuo δ_i^t , su energía E_i^t induce el covector

$$dE_i^t(s), \quad (206)$$

y mediante la métrica G_t se obtiene el vector

$$V_i^t(s) = -G_t^{-1} dE_i^t(s). \quad (207)$$

Este vector es una dirección de reorganización del estado. Salvo degeneración de la métrica o del gradiente,

$$\delta_i^t(s) \neq 0 \implies V_i^t(s) \neq 0. \quad (208)$$

Por tanto, una incoherencia visible por u/p induce una dirección operacional. A partir de V_i^t , CTNet puede definir un operador local de transformación:

$$O_i^t(s) = s + \eta V_i^t(s), \quad (209)$$

o, en forma más general,

$$O_i^t : \mathcal{S} \longrightarrow \mathcal{S}, \quad O_i^t(s) = \Phi_{\theta_t}(s, \delta_i^t(s)). \quad (210)$$

La cadena fundamental es

$$\delta_i^t(s) \longmapsto E_i^t(s) \longmapsto dE_i^t(s) \longmapsto V_i^t(s) \longmapsto O_i^t. \quad (211)$$

Esto expresa de forma directa que u/p no mide solamente una diferencia: convierte una diferencia en operador.

D. Coherencia, incoherencia y frontera de cierre

La coherencia y la incoherencia son polos informativos de una misma estructura. Si

$$\delta_i^t(s) = 0, \quad (212)$$

entonces s pertenece a la región de cierre asociada a la proyección p_i^t :

$$s \in \mathcal{A}_i^t, \quad \mathcal{A}_i^t = \{s \in \mathcal{S} : \delta_i^t(s) = 0\}. \quad (213)$$

Si

$$\delta_i^t(s) \neq 0, \quad (214)$$

entonces s no pertenece a esa región de cierre, pero define una normal, una frontera o una dirección de corrección:

$$n_i^t(s) = \frac{\delta_i^t(s)}{\|\delta_i^t(s)\|}. \quad (215)$$

Por tanto,

$$\text{coherencia} \implies \text{atractor de cierre}, \quad (216)$$

$$\text{incoherencia} \implies \text{frontera/dirección}. \quad (217)$$

Ambas aportan conocimiento. La actualización funcional puede escribirse como

$$K_{t+1} = K_t + \eta \mathcal{A}_{\theta_t}(\{\delta_i^t(s_t)\}_{i \in I_t}), \quad (218)$$

donde $\mathcal{A}_{\theta_t}$ es el operador de actualización inducido por los residuos. La coherencia estabiliza; la incoherencia orienta. La primera dice dónde cierra el sistema. La segunda dice por dónde no cierra y qué transformación sería necesaria.

E. Entrenamiento sin exigencia de coherencia previa

Sea una trayectoria

$$\tau = (s_0, x_0, s_1, x_1, \dots, s_T). \quad (219)$$

CTNet no requiere que

$$\Omega_t(s_t) = 0 \quad \forall t. \quad (220)$$

El aprendizaje se define por la acumulación de residuos convertidos en operadores:

$$\theta_T = \theta_0 + \int_0^T \mathcal{A}_{\theta_t}(s_t, \{\delta_i^t(s_t)\}_{i \in I_t}) dt. \quad (221)$$

Mientras exista residuo procesable,

$$\exists i : \delta_i^t(s_t) \neq 0, \quad (222)$$

existe señal de reorganización:

$$\Delta\theta_t \neq 0. \quad (223)$$

Por eso la incoherencia no impide el aprendizaje. Lo alimenta. El dato incoherente no es falta de información; es información sobre la insuficiencia del encuadre actual.

Si el encuadre actual viene dado por $\mathcal{P}_t$, una incoherencia fuerte puede expresarse como

$$\inf_{i \in I_t} \|s - u_i^t p_i^t(s)\| > \varepsilon. \quad (224)$$

Entonces no basta con modificar s . Puede ser necesario modificar la familia de proyecciones:

$$\mathcal{P}_{t+1} = \mathcal{P}_t \cup \Gamma_{\theta_t}(s_t, \{\delta_i^t(s_t)\}_{i \in I_t}), \quad (225)$$

donde Γ_{θ_t} genera nuevas vistas internas. Por eso la incoherencia no solo produce corrección: puede producir nueva observabilidad.

F. Observabilidad interna y generación de nuevas diferencias

El espacio observable interno en t puede definirse como la σ -álgebra generada por las proyecciones, energías y operadores disponibles:

$$\mathcal{O}_t = \sigma(\{p_i^t, E_i^t, O_i^t\}_{i \in I_t}). \quad (226)$$

Si la incoherencia genera una nueva proyección $p_j^{t+1} \notin \mathcal{P}_t$, entonces

$$\mathcal{O}_{t+1} = \sigma(\mathcal{O}_t, p_j^{t+1}) \supsetneq \mathcal{O}_t. \quad (227)$$

Así, CTNet no solo aprende más dentro del mismo espacio; aumenta el espacio de diferencias que puede detectar. La nueva observabilidad permite diferencias que antes eran invisibles:

$$\mathcal{O}_t \longrightarrow \delta_t \longrightarrow \mathcal{O}_{\delta_t} \longrightarrow \mathcal{O}_{t+1} \longrightarrow \delta_{t+1}. \quad (228)$$

Por tanto, cada reorganización puede abrir nuevas regiones de diferencia. La potencia no procede de una lista creciente de datos, sino de una secuencia de espacios de observabilidad cada vez más ricos:

$$\mathcal{O}_0 \subseteq \mathcal{O}_1 \subseteq \dots \subseteq \mathcal{O}_t \subseteq \dots. \quad (229)$$

G. Evento CTNet: aprendizaje del efecto de la experiencia

La reentrada convierte la salida en nuevo estímulo. Sea

$$y_t = A_{\eta_t}(s_t) \quad (230)$$

la acción o salida visible. La reentrada se modela como

$$\tilde{x}_{t+1} = R(y_t), \quad (231)$$

y el nuevo estado queda

$$s_{t+1} = T_{\theta_t}(s_t, x_t, \tilde{x}_{t+1}). \quad (232)$$

La salida no es terminal. Entra de nuevo al sistema como perturbación. Esto permite aplicar u/p no solo al dato externo, sino también a la consecuencia interna del propio sistema:

$$\delta_i^{t+1}(T_{\theta_t}(s_t, x_t, R(A_{\eta_t}(s_t))))). \quad (233)$$

Definimos un evento CTNet como

$$e_t = (x_t, s_t, \{p_i^t(s_t)\}_{i \in I_t}, \{\delta_i^t(s_t)\}_{i \in I_t}, y_t, R(y_t), s_{t+1}). \quad (234)$$

Un modelo clásico aprende de pares (x, y) . CTNet aprende de eventos e_t . La información de entrenamiento no está solo en x_t , sino en la deformación completa

$$s_t \mapsto s_{t+1}. \quad (235)$$

El valor funcional de un estímulo puede medirse por la magnitud de reorganización que induce:

$$\mathcal{V}_t(x_t) = d_{\mathcal{T}}(T_{\theta_{t+1}}, T_{\theta_t}), \quad (236)$$

donde $d_{\mathcal{T}}$ mide distancia entre funciones de transición. Un dato vale no por su contenido explícito, sino por la transformación que produce.

H. Capacidad funcional y crecimiento con menor peso explícito

La memoria reactiva puede escribirse como la familia de respuestas inducibles

$$\mathcal{R}_t = \{T_{\theta_t}(s, \cdot) : s \in \mathcal{S}\}. \quad (237)$$

Su capacidad no se mide por el número de registros almacenados, sino por la diversidad de respuestas inducibles. Para una distribución μ sobre estímulos, definimos una capacidad funcional local mediante jacobianos:

$$\mathcal{C}_t(\mu) = \log \det \left(I + \sigma^{-2} \int_{\mathcal{X}} J_t(x)^\top J_t(x) d\mu(x) \right), \quad (238)$$

donde

$$J_t(x) = \frac{\partial T_{\theta_t}(s, x)}{\partial x}. \quad (239)$$

Si el entrenamiento modifica T_{θ_t} de modo que aumenta el rango efectivo de J_t , entonces

$$\mathcal{C}_{t+1}(\mu) > \mathcal{C}_t(\mu), \quad (240)$$

aunque la memoria explícita disminuya. Esto expresa que el sistema puede saber más guardando menos: el dato se ha convertido en reactividad.

I. Clausura composicional de transformaciones

Sea

$$\mathcal{G}_t = \langle \{T_x^t : x \in \mathcal{X}\}, \{O_i^t : i \in I_t\}, R, A_{\eta_t} \rangle \quad (241)$$

el semigrupo generado por transiciones inducidas por estímulos, operadores u/p , reentrada y acción. La capacidad estructural no es solo $|\mathcal{S}|$, sino el conjunto de composiciones posibles:

$$\mathcal{G}_t^{(\leq n)} = \{g_k \circ \dots \circ g_1 : g_j \in \mathcal{G}_t, k \leq n\}. \quad (242)$$

La capacidad efectiva de trayectoria puede definirse como

$$\text{Cap}_t(n, \varepsilon) = \log N_\varepsilon \left(\{g(s) : g \in \mathcal{G}_t^{(\leq n)}, s \in \mathcal{S}\} \right), \quad (243)$$

donde N_ε es el número de bolas de radio ε necesarias para cubrir el conjunto alcanzable. El crecimiento depende de la clausura composicional:

$$T_b \circ T_a, \quad O_i \circ T_b \circ T_a, \quad R \circ A \circ O_j \circ O_i \circ T_a, \dots \quad (244)$$

Cada nuevo operador generado por u/p amplía $\mathcal{G}_t$:

$$\mathcal{G}_{t+1} = \langle \mathcal{G}_t, O_{\delta_t} \rangle. \quad (245)$$

Si

$$O_{\delta_t} \notin \mathcal{G}_t, \quad (246)$$

entonces

$$\mathcal{G}_{t+1} \supsetneq \mathcal{G}_t. \quad (247)$$

Esta es la infinitud estructural relevante: no procede de infinitos datos físicos, sino de la composición abierta de transformaciones.

J. Relatividad funcional: cambio de rol interno

Introducimos un conjunto de roles funcionales

$$\mathcal{R}_{\text{role}} = \{\text{dato, memoria, criterio, error, respuesta, operador, frontera, estimulo}\}. \quad (248)$$

Cada componente interno $c \in \mathcal{C}_t$ posee una asignación de rol

$$\rho_t : \mathcal{C}_t \longrightarrow \mathcal{R}_{\text{role}}. \quad (249)$$

En una arquitectura rígida, ρ_t es fija:

$$\rho_{t+1} = \rho_t. \quad (250)$$

En CTNet, el desacoplamiento u/p puede cambiar el rol de un componente:

$$\rho_{t+1} = \Psi_{\theta_t}(\rho_t, s_t, \{\delta_i^t(s_t)\}_{i \in I_t}). \quad (251)$$

Así, una salida puede convertirse en estímulo, una incoherencia en criterio, un residuo en memoria y un fallo en operador. Matemáticamente, CTNet no conserva una tipificación fija; produce una dinámica sobre los roles funcionales:

$$(c, \rho_t(c)) \mapsto (c, \rho_{t+1}(c)). \quad (252)$$

Esto explica por qué CTNet no solo cambia respuestas, sino qué cuenta como dato, memoria, error o criterio.

K. Criterios internos generados por residuos

Los criterios de aprendizaje tampoco tienen que ser externos y fijos. Sea $\mathcal{E}_t$ el conjunto de energías internas disponibles:

$$\mathcal{E}_t = \{E_i^t : i \in I_t\}. \quad (253)$$

Una diferencia no absorbida puede generar un nuevo criterio

$$E_{\text{new}}^{t+1} = \Lambda_{\theta_t}(s_t, \{\delta_i^t(s_t)\}_{i \in I_t}), \quad (254)$$

por lo que

$$\mathcal{E}_{t+1} = \mathcal{E}_t \cup \{E_{\text{new}}^{t+1}\}. \quad (255)$$

De este modo, CTNet no solo aprende dentro de una función de pérdida. Aprende a producir nuevas funciones de evaluación. El aprendizaje no queda completamente contenido en

$$\min_{\theta} L(\theta), \quad (256)$$

sino en la dinámica conjunta

$$(\theta_t, \mathcal{P}_t, \mathcal{E}_t, \rho_t) \mapsto (\theta_{t+1}, \mathcal{P}_{t+1}, \mathcal{E}_{t+1}, \rho_{t+1}). \quad (257)$$

L. Sistema dinámico extendido

La arquitectura completa puede escribirse como un sistema dinámico extendido

$$\Xi_t = (s_t, \theta_t, \mathcal{P}_t, \mathcal{E}_t, \rho_t, \mathcal{G}_t). \quad (258)$$

La evolución general es

$$\Xi_{t+1} = \mathfrak{F}(\Xi_t, x_t, \{\delta_i^t(s_t)\}_{i \in I_t}, R(A_{\eta_t}(s_t))). \quad (259)$$

El principio u/p entra porque los residuos δ_i^t no son salidas auxiliares; son argumentos de la evolución de todo el sistema. Por eso la diferencia local-global puede modificar

$$s_t, \quad \theta_t, \quad \mathcal{P}_t, \quad \mathcal{E}_t, \quad \rho_t, \quad \mathcal{G}_t. \quad (260)$$

En forma compacta:

$$\delta^t \longmapsto (\Delta s_t, \Delta \theta_t, \Delta \mathcal{P}_t, \Delta \mathcal{E}_t, \Delta \rho_t, \Delta \mathcal{G}_t). \quad (261)$$

Esta es la razón formal de la potencia de u/p : una diferencia no modifica una sola variable; puede modificar estado, aprendizaje, observabilidad, criterios, roles internos y semigrupo de transformaciones.

M. Crecimiento de conocimiento por cierre y no cierre

El crecimiento de conocimiento puede escribirse como crecimiento de capacidad futura. Decimos que

$$K_{t+1} > K_t \quad (262)$$

si existe una región $U \subseteq \mathcal{X} \times \mathcal{S}$ tal que

$$T_{\theta_{t+1}}|_U \neq T_{\theta_t}|_U \quad (263)$$

y dicha diferencia mejora cierre, discriminación, orientación o generación de operadores. Para cierres positivos, una condición directa es

$$\Omega_{t+1}(T_{\theta_{t+1}}(s, x)) < \Omega_t(T_{\theta_t}(s, x)). \quad (264)$$

Para incoherencias, la ganancia puede aparecer aunque la energía no disminuya inmediatamente, siempre que el gradiente de cierre se vuelva más informativo:

$$\|\nabla \Omega_{t+1}(T_{\theta_{t+1}}(s, x))\| > \|\nabla \Omega_t(T_{\theta_t}(s, x))\|. \quad (265)$$

Así, incluso cuando una trayectoria no cierra, puede aumentar conocimiento si produce una nueva dirección útil:

$$\delta_i^t(s_t) \neq 0 \quad \text{y} \quad O_{\delta_i^t} \notin \mathcal{G}_t. \quad (266)$$

Entonces

$$\mathcal{G}_{t+1} = \langle \mathcal{G}_t, O_{\delta_i^t} \rangle \supsetneq \mathcal{G}_t. \quad (267)$$

El no cierre añade estructura.

N. Convertibilidad funcional como categoría interna

La convertibilidad funcional total puede expresarse como una categoría interna. Sea $\mathbf{C}_t$ una categoría cuyos objetos son componentes funcionales del sistema y cuyos morfismos son transformaciones entre roles:

$$c_a \xrightarrow{m} c_b. \quad (268)$$

El principio u/p genera morfismos a partir de residuos:

$$\delta_i^t \longmapsto m_{\delta_i^t}. \quad (269)$$

Si una salida se convierte en entrada, existe un morfismo

$$y_t \xrightarrow{R} x_{t+1}. \quad (270)$$

Si un error se convierte en criterio, existe

$$\delta_i^t \xrightarrow{\Lambda} E_{\text{new}}^{t+1}. \quad (271)$$

Si una incoherencia se convierte en operador, existe

$$\delta_i^t \xrightarrow{\Phi} O_{\delta_i^t}. \quad (272)$$

Por tanto, CTNet no opera sobre objetos funcionalmente fijos. Opera sobre una categoría dinámica

$$\mathbf{C}_t \longmapsto \mathbf{C}_{t+1}, \quad (273)$$

donde los residuos generan nuevos morfismos. Esta es la expresión matemática de que CTNet transmuta dato, memoria, error, criterio y respuesta en una misma sustancia operativa.

Ñ. Resumen formal del principio

El principio completo puede resumirse como

$$\boxed{\begin{array}{l} \text{diferencia} \longrightarrow \text{residuo} \longrightarrow \text{energía} \longrightarrow \text{campo vectorial} \\ \longrightarrow \text{operador} \longrightarrow \text{nueva reactividad} \longrightarrow \text{nueva observabilidad} \end{array}} \quad (274)$$

y, recursivamente,

$$\boxed{\begin{array}{l} \mathcal{O}_t \longrightarrow \delta_t \longrightarrow \mathcal{O}_{\delta_t} \longrightarrow \mathcal{G}_{t+1} \\ \longrightarrow \mathcal{O}_{t+1} \longrightarrow \delta_{t+1} \end{array}} \quad (275)$$

La potencia de CTNet no reside en una salida concreta, sino en que el ciclo anterior no termina internamente mientras existan diferencias procesables:

$$\exists \delta_t \neq 0 \Rightarrow \exists \mathcal{O}_{\delta_t} \Rightarrow \mathcal{G}_{t+1} \supseteq \mathcal{G}_t \Rightarrow \mathcal{O}_{t+1} \supseteq \mathcal{O}_t \Rightarrow \exists \delta_{t+1} \text{ nueva.} \quad (276)$$

Cada cierre genera una región estable:

$$\delta_t = 0 \Rightarrow s_t \in \mathcal{A}_t. \quad (277)$$

Cada no cierre genera una frontera:

$$\delta_t \neq 0 \Rightarrow \partial \mathcal{A}_t \quad \text{o dirección hacia nueva región.} \quad (278)$$

Cada frontera puede generar un operador:

$$\partial \mathcal{A}_t \Rightarrow \mathcal{O}_{\partial \mathcal{A}_t}. \quad (279)$$

Cada operador modifica la clausura de transformaciones:

$$\mathcal{O}_{\partial \mathcal{A}_t} \Rightarrow \mathcal{G}_{t+1}. \quad (280)$$

Cada nueva clausura permite nuevas diferencias:

$$\mathcal{G}_{t+1} \Rightarrow \mathcal{O}_{t+1} \Rightarrow \delta_{t+1}. \quad (281)$$

Por eso CTNet no solo aprende respuestas. Aprende condiciones de aparición de nuevas respuestas. No solo aprende dentro de un espacio; aprende a modificar el espacio de comparación. No solo acumula memoria; convierte memoria en forma de lectura. No solo reduce

error; convierte error en operador. No solo utiliza coherencia; utiliza también incoherencia como negativo generador de cierre.

En forma compacta,

$$\boxed{\text{CTNet} = (\mathcal{S}, T_\theta, \mathcal{P}, u, \Omega, \mathcal{G}, R, A)} \quad (282)$$

con evolución extendida

$$\boxed{\Xi_{t+1} = \mathfrak{F}(\Xi_t, x_t, \delta_t, R(A(s_t)))} \quad (283)$$

donde

$$\boxed{\delta_t = \{s_t - u_i^t p_i^t(s_t)\}_{i \in I_t}} \quad (284)$$

no es un error auxiliar, sino el generador de transformación:

$$\boxed{\delta_t \longmapsto (\Delta s_t, \Delta \theta_t, \Delta \mathcal{P}_t, \Delta \mathcal{E}_t, \Delta \rho_t, \Delta \mathcal{G}_t)} \quad (285)$$

Esta es la razón formal de su potencia: u/p convierte cualquier diferencia entre parte y totalidad en modificación posible de estado, memoria, criterio, observabilidad, roles y operadores. CTNet no aprende datos; aprende a transformar el sistema que hace que los datos puedan tener significado operativo.

XLI. LECTURA CONVENCIONAL FINAL DEL MECANISMO

El mecanismo completo puede resumirse mediante una composición:

$$x \xrightarrow{\text{Enc}} \Xi_0 \xrightarrow{\Phi_\theta^T} \Xi_T \xrightarrow{Q} (\omega, s) \xrightarrow{G} y \xrightarrow{\text{Enc}} \Xi_y. \quad (286)$$

La trayectoria no necesita ser coherente para ser entrenable. El sistema registra tanto las trayectorias que cierran como las que no cierran:

$$\tau \longmapsto (E_{\text{coh}}(\tau), \omega(\tau), s(\tau), \Delta F(\tau)). \quad (287)$$

Las trayectorias coherentes estabilizan regiones de respuesta; las incoherentes definen contraste, frontera y dirección de reorganización. En varias proyecciones se mide

$$d_k(\Pi_k(\Xi_T), \Pi'_k(\Xi_T)), \quad k = 1, \dots, K, \quad (288)$$

pero esa medida no es una condición previa de admisión. Es una coordenada de aprendizaje. Si la salida reingresada altera el cierre, esa alteración también queda como información sobre la función de respuesta:

$$\Delta Q_y = Q(\Phi_\theta(\Xi_y)) - Q(\Xi_T). \quad (289)$$

Cuando existe una tarea formal verificable, la respuesta cerrada debe satisfacer

$$V(x, y) = 1. \quad (290)$$

Cuando no lo satisface, el fallo no es vacío: queda como caso negativo verificable, actualiza la frontera de decisión y puede inducir un nuevo operador o una nueva ruta de respuesta.

En esta formulación, CTNet es una arquitectura de procesamiento reactivo: la información entrante no solo se representa, sino que modifica el modo de respuesta del sistema. La memoria crece como competencia funcional, y también crece al registrar patrones de no cierre, contraejemplos y trayectorias incoherentes que aumentan la perspectiva disponible para futuras entradas.

XLII. CONCLUSIÓN

CTNet puede formularse convencionalmente como una arquitectura neuronal dinámica de estado latente estructurado. Sus rasgos principales son: empaquetado de observaciones heterogéneas en un estado único, transformación reversible o casi reversible, memoria funcional codificada en la reactividad del sistema, aprendizaje tanto de trayectorias coherentes como incoherentes, regularización de coherencia multivista, diagnóstico interno de error no absorbido, generación de operadores a partir de residuos local-global, memoria procedimental direccionable por clave, autoobservación de procesos internos, reentrada de la salida generada y resolución verificable mediante operadores de dominio.

La frase técnica más compacta es la siguiente: CTNet no recuerda principalmente porque almacene más registros, sino porque modifica cómo responde. Su memoria efectiva está en la función de transición, el paisaje de atractores, las rutas de activación y las reglas recuperables. Su salida no es solo predicción; es una acción observable que vuelve a entrar al sistema para ser evaluada como parte de la dinámica.

El funcionamiento total puede resumirse como una arquitectura de aprendizaje y resolución en bucle cerrado: una observación perturba el estado, el estado se transforma conservando información, el sistema mide la coherencia o incoherencia de esa transformación, produce una acción, reobserva la acción, ajusta su memoria funcional y, cuando la tarea es formal, exige verificación externa. Este diseño permite que la capacidad crezca por reactividad, compresión funcional, adquisición de operadores verificables y aprovechamiento de ejemplos

negativos o incoherentes, sin depender de una acumulación lineal de registros explícitos.

REFERENCIAS

1. L. Dinh, D. Krueger, and Y. Bengio, “NICE: Non-linear Independent Components Estimation,” *arXiv:1410.8516* (2014).
2. A. N. Gomez, M. Ren, R. Urtasun, and R. B. Grosse, “The Reversible Residual Network: Backpropagation Without Storing Activations,” *Advances in Neural Information Processing Systems* **30** (2017).
3. B. Chang, L. Meng, E. Haber, F. Tung, and D. Begert, “Reversible Architectures for Arbitrarily Deep Residual Neural Networks,” *AAAI Conference on Artificial Intelligence* (2018).
4. J. J. Hopfield, “Neural networks and physical systems with emergent collective computational abilities,” *Proceedings of the National Academy of Sciences* **79**, 2554–2558 (1982).
5. A. Graves et al., “Hybrid computing using a neural network with dynamic external memory,” *Nature* **538**, 471–476 (2016).
6. A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” *arXiv:1807.03748* (2018).